


Clase 1: SystemVerilog

Diseño e Implementación de Circuitos Digitales en VLSI

Agustín Pesce - José Torres - Santiago Garaventa— Becas de Grado 2026

Mayo 2026

Contents

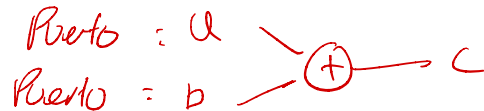
1	Introducción	2
2	Verilog vs SystemVerilog	2
2.1	¿Por qué cambiamos?	2
2.2	wire/reg vs logic	2
2.3	always vs always_ff / always_comb	3
3	Tipos de datos	3
3.1	Tipos básicos	3
3.2	Constantes y literales	3
3.3	Packed arrays (vectores de bits)	4
3.4	Unpacked arrays (arreglos de elementos)	4
3.5	Packed vs Unpacked: comparación	5
3.6	typedef enum: estados con nombre	5
3.7	typedef struct packed: agrupar señales	6
4	Lógica combinacional	6
4.1	assign: conexiones permanentes	6
4.2	Operadores	6
4.3	Ejemplo: Half Adder	6
4.4	Ejemplo: Full Adder	7
4.5	Ripple Carry Adder	8
4.6	always_comb: combinacional complejo	8
5	Lógica secuencial	9
5.1	El Flip-Flop	9
5.2	always_ff: describiendo flip-flops	9
5.3	Reset síncrono vs asíncrono	10
6	Modules y jerarquía	10
6.1	Anatomía de un module	10
6.2	Instanciación	10
6.3	generate: crear hardware con loops	11
7	Blocking vs Non-blocking	11
8	Resumen	12

1 Introducción

SystemVerilog es el lenguaje estándar de la industria para diseñar y verificar circuitos digitales. Nació como extensión de Verilog y hoy lo reemplazó por completo en todos los flujos modernos de ASIC y FPGA.

La diferencia fundamental con un lenguaje de software como C o Python es que SystemVerilog **describe hardware**: no es una secuencia de instrucciones que se ejecutan una tras otra, sino una descripción de circuitos que existen **todos al mismo tiempo** en silicio.

Cuando escribís `assign c = a + b;` no le estás diciendo a un procesador “calculá a+b”. Estás diciendo “existe un sumador físico conectado entre las señales a, b, y c”. Ese sumador está ahí permanentemente, funcionando todo el tiempo.



2 Verilog vs SystemVerilog

2.1 ¿Por qué cambiamos?

Verilog se creó en 1984. En 2005 apareció SystemVerilog como un superset que agrega mejor tipado, nuevas construcciones, y soporte para verificación. Hoy en la industria todo diseño nuevo se escribe en SystemVerilog.

Las diferencias principales:

Feature	Verilog	SystemVerilog
Tipos de datos	wire, reg, integer	logic, bit, int, byte
Asignación	always @(...)	always_ff, always_comb
Constantes	8'd0	'0, '1 (auto-width)
Enumeraciones	No existe	typedef enum
Structs	No existe	typedef struct packed
Arrays	Básicos	Packed + unpacked
Interfaces	No existe	interface
Clases (verif.)	No existe	class, OOP completo

Handwritten notes: `logic [7:0] a`, `always_ff(L)..`, `a <= 'b0`

2.2 wire/reg vs logic

Handwritten note: `Ldata 64 ancho - 8bits`, `data [64x8-10]`

En Verilog tenías que elegir entre **wire** (para `assign`) y **reg** (para `always`). Si elegías mal, error de compilación. El nombre **reg** era confuso porque no siempre generaba un registro (flip-flop) — era solo un requisito sintáctico.

Verilog (viejo):

```
wire [7:0] suma;
assign suma = a + b; // OK

reg [7:0] count;
always @(posedge clk)
    count <= count + 1; // OK

reg [7:0] x;
assign x = a + b; // ERROR!
```

SystemVerilog (nuevo):

```
logic [7:0] suma;
assign suma = a + b; // OK

logic [7:0] count;
always_ff @(posedge clk)
    count <= count + 1; // OK

logic [7:0] x;
assign x = a + b; // OK
```

logic reemplaza a ambos. El compilador deduce si es wire o registro según cómo lo uses. No hay más confusión.

Handwritten notes: `input logic [8:0] L-data [64:1:0]`, `L-data [23]`

module filter (

input logic signed [8-1:0] coef [16-1:0],

input clk,

output logic [8-1:0] o_coef [16-1:0]

{

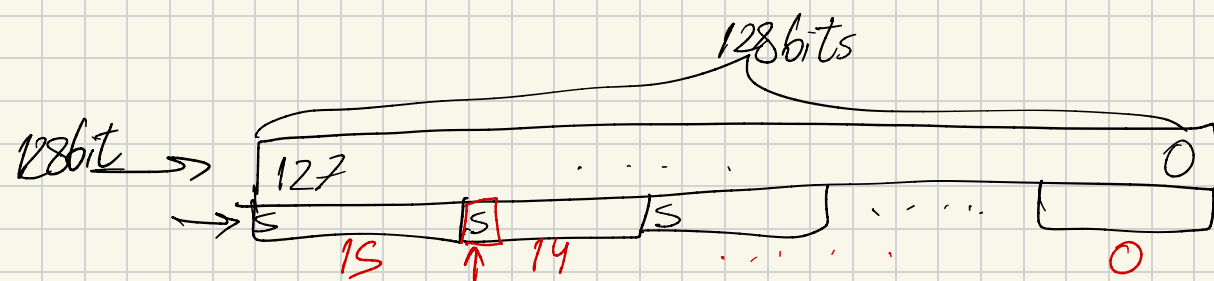


input wire [16.8-1:0] coef_v 16 x 8 = 128.

type def struct { logic signed [8-1:0] coef } coef_t [2] 0

input coef_t [16-1:0] coef_sv;

16 → coef signed



assign coef_sv = coef

$\text{coef_sv}[14][7] = 16\text{bit}$

2.3 always vs always_ff / always_comb

En Verilog, el bloque `always` servía para todo: combinacional y secuencial. No había forma de que el compilador supiera tu intención. Si te olvidabas una señal en la lista de sensibilidad, el diseño simulaba diferente a como sintetizaba.

Verilog:

```
// Secuencial
always @(posedge clk)
    q <= d;

// Combinacional (lista manual!)
always @(a or b)
    c = a & b;

// Bug comun: falta b
always @(a)          // OOPS
    c = a & b;       // simula mal
```

SystemVerilog:

```
// Secuencial (explicito)
always_ff @(posedge clk)
    q <= d;

// Combinacional (automatico!)
always_comb
    c = a & b;
// Sensibilidad completa,
// siempre

// El compilador chequea:
always_ff @(posedge clk)
    c = a & b; // WARNING: =
```

Regla

Nunca usar `always` genérico en código nuevo. Siempre `always_ff` (secuencial) o `always_comb` (combinacional).

3 Tipos de datos

3.1 Tipos básicos

`logic` es el tipo universal para diseño RTL. Puede valer 0, 1, x (desconocido), o z (alta impedancia):

```
logic      flag;           // 1 bit
logic [7:0] dato;          // 8 bits (bit 7 = MSB, bit 0 = LSB)
logic [31:0] addr;         // 32 bits
```

`bit` es el tipo para testbench/verificación. Solo vale 0 o 1 (2 estados, más eficiente en simulación):

```
bit        flag;           // 1 bit, solo 0/1
bit [7:0]   dato;          // 8 bits
```

Tipos enteros para variables de software (testbench, parámetros):

```
int         x;             // 32 bits, signed (-2G..2G)
byte        b;             // 8 bits, signed (-128..127)
shortint    s;             // 16 bits, signed
longint     l;             // 64 bits, signed
```

3.2 Constantes y literales

El formato es `<ancho>'<base><valor>`:

```
// Binario (b)
1'b0           // 1 bit, valor 0
```

```

1'b1           // 1 bit, valor 1
4'b1010       // 4 bits = 10 decimal
8'b0000_1111  // guion bajo para legibilidad

// Decimal (d)
8'd255        // 8 bits, valor 255
32'd42        // 32 bits, valor 42

// Hexadecimal (h)
8'hFF         // 8 bits = 255
8'hA5         // 8 bits = 165
32'hDEAD_BEEF // 32 bits

// Especiales de SystemVerilog (auto-width)
'0            // todos ceros (se adapta al ancho)
'1            // todos unos
'x            // todos X
'z            // todos Z

```

Tip

'0 y '1 son muy útiles porque se adaptan automáticamente al ancho de la señal. `count <= '0`; pone en cero un contador de cualquier ancho sin tener que escribir `8'd0`.

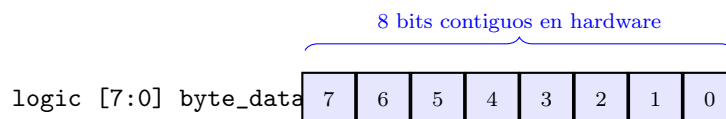
3.3 Packed arrays (vectores de bits)

Un **packed array** es un vector contiguo de bits. Las dimensiones van **a la izquierda** del nombre:

```

logic [7:0]    byte_data; // 8 bits contiguos
logic [31:0]   word;      // 32 bits contiguos
logic [3:0][7:0] data;    // 4 bytes = 32 bits contiguos

```



Con packed arrays multidimensionales podés acceder por byte o por bit:

```

logic [3:0][7:0] data; // 4 bytes, 32 bits totales
data[2] = 8'hFF;       // acceder al byte 2
data[1][3:0] = 4'hA;   // acceder a 4 bits del byte 1
logic [31:0] flat = data; // asignar como vector plano (32 bits)
data = data + 1;        // operar como un numero de 32 bits

```

Los packed arrays generan **cables o registros** en el hardware — son lo que usás para señales, buses, y datos.

3.4 Unpacked arrays (arreglos de elementos)

Un **unpacked array** es un arreglo de elementos separados. Las dimensiones van **a la derecha** del nombre:

```

logic [7:0] mem [0:255]; // 256 elementos de 8 bits cada uno
logic [7:0] fifo [0:15]; // 16 elementos de 8 bits

```

[255:0]

```
int      scores [10];      // 10 enteros
```

Cada elemento vive separado. No se puede operar como un solo vector:

```
mem[0] = 8'hFF;           // OK: acceder a un elemento
mem[1] = mem[0] + 1;      // OK: operar elementos individuales
logic [31:0] f = mem;     // ERROR: no se puede aplanar
```

Los unpacked arrays generan **memorias** en el hardware (RAMs, register files).

3.5 Packed vs Unpacked: comparación

	Packed	Unpacked
Dimensiones	A la izquierda	A la derecha
Ejemplo	logic [3:0][7:0] d;	logic [7:0] d [4];
Almacenamiento	Bits contiguos	Elementos separados
Asignar a vector	Sí	No
Operaciones	Como número completo	Elemento por elemento
Genera en HW	Cables, registros	Memorias
Uso típico	Señales, buses	RAMs, FIFOs, register files

Combinando packed + unpacked

Es muy común combinarlos para describir un register file:

```
logic [7:0] register_file [0:31]; // 32 registros de 8 bits
//      ^^^^^      ^^^^^
//      packed (ancho)      unpacked (cantidad)

register_file[5] = 8'hA5;           // escribir registro 5
register_file[5][3:0] = 4'hF;       // escribir bits 3:0 del
//      registro 5
```

3.6 typedef enum: estados con nombre

En vez de usar números mágicos para estados de una FSM (máquina de estados):

```
typedef enum logic [1:0] {
    IDLE = 2'b00,
    RUN  = 2'b01,
    DONE = 2'b10
} state_t;

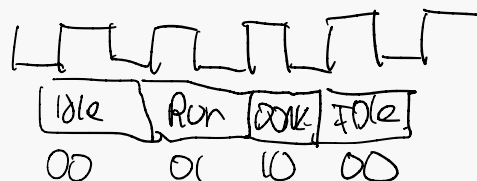
state_t state;

always_ff @(posedge clk)
    case (state)
        IDLE: state <= RUN;
        RUN:  state <= DONE;
        DONE: state <= IDLE;
    endcase
```

00
01
10
11

// defino un nuevo tipo

// variable de ese tipo



Si asignás un valor no definido en el enum, el compilador te avisa. Con localparam esto no pasaba.

3.7 typedef struct packed: agrupar señales

Un struct packed agrupa señales relacionadas en un tipo con campos nombrados:

```
typedef struct packed {
    logic [7:0] opcode;    // bits [15:8]
    logic [3:0] rs;        // bits [7:4]
    logic [3:0] rd;        // bits [3:0]
} instruction_t;

instruction_t instr;
instr.opcode = 8'hAB;      // acceder por nombre
instr.rs = 4'd3;
logic [15:0] flat = instr; // usar como vector de 16 bits
```

Handwritten notes:
 assign ~
 assign ~
 → creo la estructura

Es packed, por lo tanto los bits son contiguos y es completamente sintetizable.

4 Lógica combinacional

La lógica combinacional no tiene clock ni memoria. La salida depende **únicamente** de las entradas actuales.

4.1 assign: conexiones permanentes

assign crea un cable con lógica. Si una entrada cambia, la salida cambia instantáneamente:

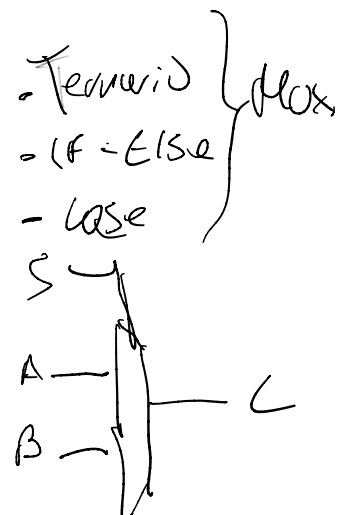
```
assign c = a & b;          // AND
assign d = a | b;          // OR
assign e = a ^ b;          // XOR
assign f = ~a;             // NOT (inversion)
assign g = a + b;          // Sumador
assign h = sel ? a : b;    // Multiplexor 2:1
assign i = {a, b};         // Concatenacion
assign {msb, lsb} = dato;  // Split
```

4.2 Operadores

Categoría	Operador	Nombre	Ejemplo
Bitwise	&	AND	c = a & b
		OR	c = a b
	^	XOR	c = a ^ b
	~	NOT	c = ~a
Aritmético	+ - *	Suma, resta, mult.	c = a + b
Shift	« »	Shift izq/der	c = a « 2
Comparación	== != < >	Igualdad, orden	c = (a == b)
Lógico	&&	AND/OR lógico	if (a && b)
Reducción	& ^	Reduce a 1 bit	c = &a
Condicional	? :	Ternario (mux)	c = s ? a : b
Concatenar	{ }	Unir señales	c = {a, b}
Replicar	{ { } }	Repetir	c = {4{a}}

4.3 Ejemplo: Half Adder

Un half adder suma dos bits y produce un resultado (sum) y un carry:



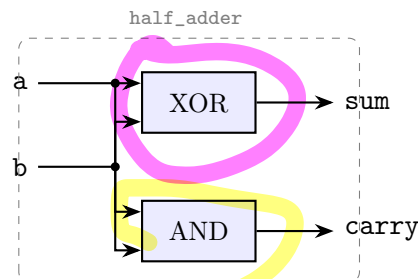
```

module half_adder (
    input  logic a,
    input  logic b,
    output logic sum,
    output logic carry
);
    assign sum    = a ^ b;    // XOR
    assign carry  = a & b;    // AND
endmodule

```

Tabla de verdad:

a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Limitación: no tiene entrada de carry, por lo tanto no se puede encadenar para sumar números de más de 1 bit.

4.4 Ejemplo: Full Adder

Un **full adder** agrega cin (carry de entrada), lo que permite encadenar varios para sumar números de N bits:

```

module full_adder (
    input  logic a,
    input  logic b,
    input  logic cin,
    output logic sum,
    output logic cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (a & cin) | (b & cin);
endmodule

```

La tabla de verdad completa tiene $2^3 = 8$ filas:

a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Un full adder se puede construir con 2 half adders + 1 OR:

```

module full_adder_structural (
    input logic a, b, cin,
    output logic sum, cout

```

```

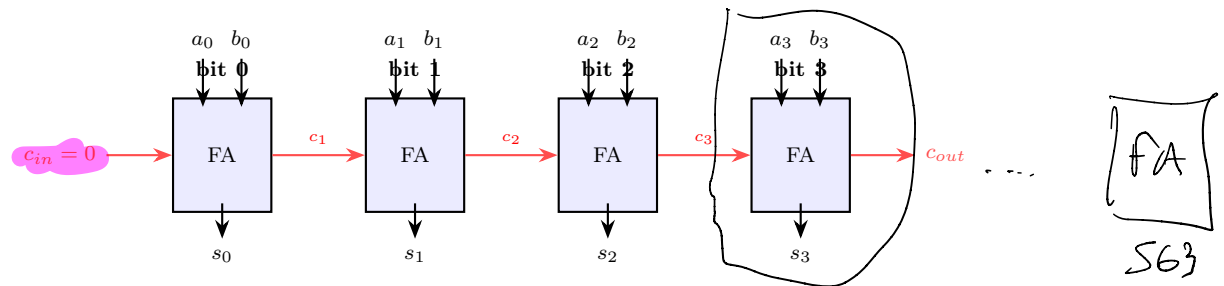
);
logic p, g, c_ha2;
half_adder ha1 (.a(a), .b(b), .sum(p), .carry(g));
half_adder ha2 (.a(p), .b(cin), .sum(sum), .carry(c_ha2));
assign cout = g | c_ha2;
endmodule

```

4.5 Ripple Carry Adder

2 4bits + 6 4bits

Encadenando el cout de un FA al cin del siguiente armamos un sumador de N bits:



```

module adder_4bit (
    input logic [3:0] a, b,
    input logic      cin,
    output logic [3:0] sum,
    output logic      cout
);
logic [3:1] c;
full_adder fa0 (.a(a[0]), .b(b[0]), .cin(cin), .sum(sum[0]), .cout(c[1]));
full_adder fa1 (.a(a[1]), .b(b[1]), .cin(c[1]), .sum(sum[1]), .cout(c[2]));
full_adder fa2 (.a(a[2]), .b(b[2]), .cin(c[2]), .sum(sum[2]), .cout(c[3]));
full_adder fa3 (.a(a[3]), .b(b[3]), .cin(c[3]), .sum(sum[3]), .cout(cout));
endmodule

```

64 bits F.L.T.O

El problema del ripple carry: el carry se propaga bit a bit. Para 32 bits, el carry del bit 0 tiene que pasar por 32 FAs antes de llegar al bit 31. Eso es **lento**.

En la práctica usamos el operador + y el sintetizador elige la implementación óptima:

```

assign {cout, sum} = a + b + cin; // el sintetizador hace la magia

```

4.6 always_comb: combinacional complejo

Para lógica que es difícil de expresar con assign (como decoders o priority encoders), usás always_comb:

```

// Decoder 2:4
always_comb begin
    case (sel)
        2'b00: out = 4'b0001;
        2'b01: out = 4'b0010;
        2'b10: out = 4'b0100;
        2'b11: out = 4'b1000;
    endcase
end

```

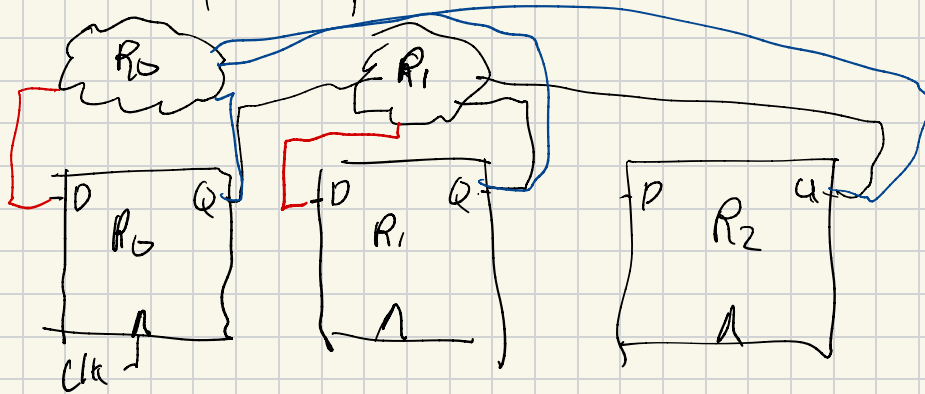
4 → 1

3 bits

	Actual	Siguiente
	R_0	R_0^+
	0 0 0	0 0 1
	0 0 1	0 1 0
	0 1 0	0 1 1
①	0 1 1	1 0 0
②	1 0 0	1 0 1
	1 0 1	1 1 0
	1 1 0	1 1 1
	1 1 1	0 0 0

contador $\leftarrow$ contador + 1

$$R_0 \equiv \bar{R}_0 R_1 R_2 + R_0 \bar{R}_1 \bar{R}_2 + R_0 \bar{R}_1 R_2 + R_0 R_1 \bar{R}_2$$



IF $((A \text{ and } B) \vee (C) \wedge D)$ begin
1

logic control

control $\leftarrow ((A \text{ and } B) \vee (C) \wedge D)$;

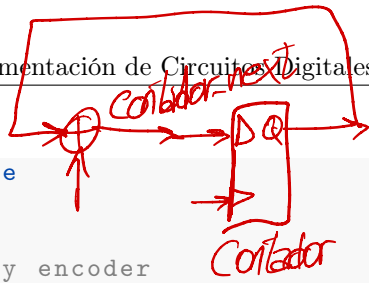
IF (control) begin

```

endcase
end

// Priority encoder
always_comb begin
    if (req[3])    grant = 2'd3;
    else if (req[2]) grant = 2'd2;
    else if (req[1]) grant = 2'd1;
    else          grant = 2'd0;
end

```



~~always_comb begin~~
~~contador.next = contador + 1;~~
~~end~~

always_ff @(posedge clk) begin
 if (reset) begin
 contador <= 'b0;
end
 else begin
 contador <= contador + 1;
end
 end

Cuidado con latches

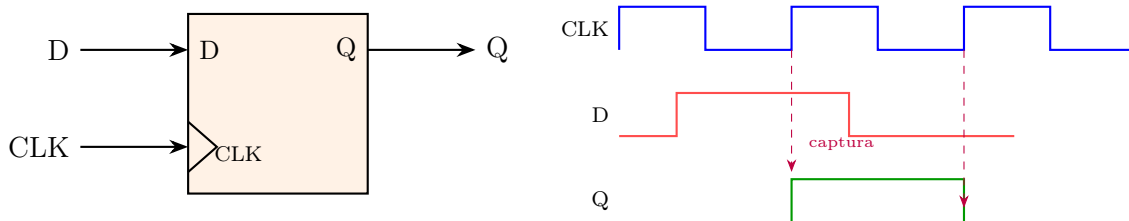
En `always_comb`, todos los casos deben estar cubiertos. Si un case no tiene default, o un if no tiene else, el sintetizador infiere un **latch** — casi siempre un error.

5 Lógica secuencial

La lógica secuencial tiene **memoria**: la salida depende del estado anterior. Se implementa con flip-flops que se actualizan en el flanco del clock.

5.1 El Flip-Flop

Un flip-flop captura el valor de su entrada D en el flanco positivo del clock, y lo mantiene en Q hasta el próximo flanco:



Q sigue a D, pero **con un retraso de un ciclo de clock**. Esa es la esencia de la lógica secuencial.

5.2 always_ff: describiendo flip-flops

```

// Flip-flop simple
always_ff @(posedge clk)
    q <= d;

// Con reset sincrono y enable
always_ff @(posedge clk)
    if (rst)    q <= '0;           // reset tiene prioridad
    else if (enable) q <= d;       // si no, capturar D
    // sin else: q mantiene su valor (no hace falta escribirlo)

```

Cada señal asignada con `<=` dentro de `always_ff` genera flip-flops: `logic [7:0] count` genera 8 flip-flops.

5.3 Reset síncrono vs asíncrono

Reset síncrono:

```
always_ff @(posedge clk) begin
    if (rst)
        count <= '0;
    else if (enable)
        count <= count + 1;
end
```

El reset **solo actúa en el posedge del clock**. Necesita que el clock esté corriendo. Es más limpio, no genera glitches. Es el default en la mayoría de diseños.

Reset asíncrono:

```
always_ff @(posedge clk or
            posedge rst) begin
    if (rst)
        count <= '0;
    else if (enable)
        count <= count + 1;
end
```

El reset actúa **inmediatamente**, sin esperar el clock. Funciona aunque el clock esté parado. Se usa cuando no hay garantía de clock al encender (power-on reset).

La diferencia en la síntesis: el reset síncrono pasa por un MUX antes del flip-flop. El reset asíncrono va directo al pin CLR del flip-flop.

En la industria

Lo más común es **asíncrono assert + síncrono deassert**: el reset se activa inmediatamente (funciona sin clock), pero se suelta sincronizado con el clock (evita metaestabilidad). Se implementa con un “reset synchronizer” de 2 flip-flops.

6 Modules y jerarquía

6.1 Anatomía de un module

Un module es la unidad de diseño. Tiene parámetros (opcionales), ports (entradas/salidas), señales internas, y lógica:

```
module contador #(
    parameter int WIDTH = 8          // parametro configurable
) (
    input  logic      i_clk,          // entrada: clock
    input  logic      i_rst,          // entrada: reset
    input  logic      i_enable,       // entrada: enable
    output logic [WIDTH-1:0] o_count // salida: valor
);
    logic [WIDTH-1:0] count_reg;      // senal interna

    always_ff @(posedge i_clk)
        if (i_rst) count_reg <= '0;
        else if (i_enable) count_reg <= count_reg + 1'b1;

    assign o_count = count_reg;       // conectar salida
endmodule
```

6.2 Instanciación

Para usar un module dentro de otro, lo instanciás y conectás:

```
module top (
    input  logic      clk, rst,
    output logic [7:0] count_a, count_b
```

```

);
// Dos contadores independientes
contador #(.WIDTH(8)) cnt_a (
    .i_clk(clk), .i_rst(rst), .i_enable(1'b1), .o_count(count_a)
);
contador #(.WIDTH(8)) cnt_b (
    .i_clk(clk), .i_rst(rst), .i_enable(count_a[7]), .o_count(
        count_b)
);
endmodule

```

Cada instancia es independiente con su propia copia de flip-flops. `cnt_a` y `cnt_b` no comparten estado.

6.3 generate: crear hardware con loops

```

module adder_Nbit #(parameter int N = 8) (
    input  logic [N-1:0] a, b,
    input  logic          cin,
    output logic [N-1:0] sum,
    output logic          cout
);
    logic [N:0] c;
    assign c[0] = cin;

    genvar i;
    generate
        for (i = 0; i < N; i++) begin : gen_fa
            full_adder fa (
                .a(a[i]), .b(b[i]), .cin(c[i]),
                .sum(sum[i]), .cout(c[i+1])
            );
        end
    endgenerate

    assign cout = c[N];
endmodule

```

El `generate` se resuelve en **tiempo de compilación** (no en runtime). Si `N=32`, genera 32 instancias de `full_adder` con todas las conexiones.

7 Blocking vs Non-blocking

Esta es la diferencia más importante y la fuente de más bugs en diseños digitales.

Blocking (=):

Se ejecuta inmediatamente. La siguiente línea ve el valor nuevo.

```
always_comb begin
    temp = a + b;      // temp
                        se actualiza YA
    result = temp * 2; // usa el
                      nuevo temp
end
```

Usar en: `always_comb`

Non-blocking (<=):

Se programa para el final del time step. Todas las asignaciones ven los valores viejos.

```
always_ff @(posedge clk) begin
    a <= b;      // a toma el
                viejo b
    b <= a;      // b toma el
                viejo a
    // SWAP correcto!
end
```

Usar en: `always_ff`

¿Por qué importa? Porque modela el hardware real. En un chip, todos los flip-flops cambian *al mismo tiempo* en el posedge del clock. Ninguno ve el valor nuevo del otro hasta el próximo ciclo. `<=` modela exactamente eso.

Si usás `=` en `always_ff`, el swap del ejemplo de arriba daría `a=5, b=5` (bug) en vez de `a=5, b=3` (correcto).

Regla de oro (nunca romper)

- `always_ff` → siempre `<=` (non-blocking)
- `always_comb` → siempre `=` (blocking)
- Nunca mezclar `=` y `<=` en el mismo bloque

8 Resumen

Concepto	Construcción
Cable con lógica (combinacional)	<code>assign c = a + b;</code>
Bloque combinacional complejo	<code>always_comb begin ... end</code>
Flip-flops (secuencial)	<code>always_ff @(posedge clk)</code>
Reset síncrono	<code>if (rst) ... else ... dentro de always_ff</code>
Reset asíncrono	<code>always_ff @(posedge clk or posedge rst)</code>
Tipo universal	<code>logic [N-1:0] señal;</code>
Constante auto-width	<code>'0, '1</code>
Packed array	<code>logic [3:0][7:0] data; (izquierda)</code>
Unpacked array	<code>logic [7:0] mem [256]; (derecha)</code>
Enumeración	<code>typedef enum logic [1:0] {...} tipo;</code>
Struct	<code>typedef struct packed {...} tipo;</code>
Instanciación	<code>modulo #(PARAM(val)) inst (.port(señal));</code>

Para la próxima clase

1. Conectarse a la EC2 por SSH
2. Clonar el repo y crear su carpeta personal
3. Correr `python3 create_project.py` contador
4. Leer `rtl/contador.sv` — identificar cada construcción que vimos
5. Correr `make sim` y ver el resultado
6. **Desafío:** escribir un `full_adder.sv` y un `adder_4bit.sv`

Blocking (=):

Se ejecuta inmediatamente. La siguiente línea ve el valor nuevo.

```
always_comb begin
    temp = a + b;      // temp
                        // se actualiza YA
    result = temp * 2; // usa el
                      // nuevo temp
end
```

Usar en: `always_comb`

Non-blocking (<=):

Se programa para el final del time step. Todas las asignaciones ven los valores viejos.

```
always_ff @(posedge clk) begin
    a <= b;      // a toma el
                // viejo b
    b <= a;      // b toma el
                // viejo a
    // SWAP correcto!
end
```

Usar en: `always_ff`

¿Por qué importa? Porque modela el hardware real. En un chip, todos los flip-flops cambian *al mismo tiempo* en el posedge del clock. Ninguno ve el valor nuevo del otro hasta el próximo ciclo. `<=` modela exactamente eso.

Si usás `=` en `always_ff`, el swap del ejemplo de arriba daría `a=5, b=5` (bug) en vez de `a=5, b=3` (correcto).

Regla de oro (nunca romper)

- `always_ff` → siempre `<=` (non-blocking)
- `always_comb` → siempre `=` (blocking)
- Nunca mezclar `=` y `<=` en el mismo bloque

8 Resumen

Concepto	Construcción
Cable con lógica (combinacional)	<code>assign c = a + b;</code>
Bloque combinacional complejo	<code>always_comb begin ... end</code>
Flip-flops (secuencial)	<code>always_ff @(posedge clk)</code>
Reset síncrono	<code>if (rst) ... else ... dentro de always_ff</code>
Reset asíncrono	<code>always_ff @(posedge clk or posedge rst)</code>
Tipo universal	<code>logic [N-1:0] señal;</code>
Constante auto-width	<code>'0, '1</code>
Packed array	<code>logic [3:0][7:0] data; (izquierda)</code>
Unpacked array	<code>logic [7:0] mem [256]; (derecha)</code>
Enumeración	<code>typedef enum logic [1:0] {...} tipo;</code>
Struct	<code>typedef struct packed {...} tipo;</code>
Instanciación	<code>modulo #(PARAM(val)) inst (.port(señal));</code>

Para la próxima clase

1. Conectarse a la EC2 por SSH
2. Clonar el repo y crear su carpeta personal
3. Correr `python3 create_project.py` contador
4. Leer `rtl/contador.sv` — identificar cada construcción que vimos
5. Correr `make sim` y ver el resultado
6. **Desafío:** escribir un `full_adder.sv` y un `adder_4bit.sv`

Blocking (=):

Se ejecuta inmediatamente. La siguiente línea ve el valor nuevo.

```
always_comb begin
    temp = a + b;      // temp
                        // se actualiza YA
    result = temp * 2; // usa el
                      // nuevo temp
end
```

Usar en: `always_comb`

Non-blocking (<=):

Se programa para el final del time step. Todas las asignaciones ven los valores viejos.

```
always_ff @(posedge clk) begin
    a <= b;      // a toma el
                // viejo b
    b <= a;      // b toma el
                // viejo a
    // SWAP correcto!
end
```

Usar en: `always_ff`

¿Por qué importa? Porque modela el hardware real. En un chip, todos los flip-flops cambian *al mismo tiempo* en el posedge del clock. Ninguno ve el valor nuevo del otro hasta el próximo ciclo. `<=` modela exactamente eso.

Si usás `=` en `always_ff`, el swap del ejemplo de arriba daría `a=5, b=5` (bug) en vez de `a=5, b=3` (correcto).

Regla de oro (nunca romper)

- `always_ff` → siempre `<=` (non-blocking)
- `always_comb` → siempre `=` (blocking)
- Nunca mezclar `=` y `<=` en el mismo bloque

8 Resumen

Concepto	Construcción
Cable con lógica (combinacional)	<code>assign c = a + b;</code>
Bloque combinacional complejo	<code>always_comb begin ... end</code>
Flip-flops (secuencial)	<code>always_ff @(posedge clk)</code>
Reset síncrono	<code>if (rst) ... else ... dentro de always_ff</code>
Reset asíncrono	<code>always_ff @(posedge clk or posedge rst)</code>
Tipo universal	<code>logic [N-1:0] señal;</code>
Constante auto-width	<code>'0, '1</code>
Packed array	<code>logic [3:0][7:0] data; (izquierda)</code>
Unpacked array	<code>logic [7:0] mem [256]; (derecha)</code>
Enumeración	<code>typedef enum logic [1:0] {...} tipo;</code>
Struct	<code>typedef struct packed {...} tipo;</code>
Instanciación	<code>modulo #(PARAM(val)) inst (.port(señal));</code>

Para la próxima clase

1. Conectarse a la EC2 por SSH
2. Clonar el repo y crear su carpeta personal
3. Correr `python3 create_project.py` contador
4. Leer `rtl/contador.sv` — identificar cada construcción que vimos
5. Correr `make sim` y ver el resultado
6. **Desafío:** escribir un `full_adder.sv` y un `adder_4bit.sv`